

TiendaControl Backend

SUMMARY

Build the backend for TiendaControl, an offline-first POS and inventory platform designed for Colombian tiendas de barrio and minimercados. Primary objective: Build a production-grade backend while learning modern backend engineering practices.

PROJECTS

TiendaControl Backend | Offline-first POS / Inventory / Financial Ledger SaaS backend engineering project.

- Stack: TypeScript + Fastify + PostgreSQL + Docker
- Architecture: Modular Monolith + Event-Sourced Synchronization
- Supports multi-tenant stores, users/authentication, products/inventory, sales, cash shifts, petty cash, customer credit (fiado), immutable financial/domain events, offline client synchronization, inventory calculations, idempotent event processing, auditability, async jobs, API documentation, automated testing, containerized development, CI/CD, logging/observability, and production security/configuration
- Implements event-sourced delta ledger synchronization with immutable events (UUID + timestamp) processed sequentially per tenant
- Designed around correctness, data integrity, concurrency, security, maintainability, testability, observability, reliability, deployment, and scalability

TECHNICAL SKILLS

TypeScript

Fastify

PostgreSQL

Docker

Modular Monolith

Event-Sourced Synchronization

Offline Synchronization

Event Sourcing

Idempotent event processing

Concurrency control

Authentication

Authorization

RBAC

Multi-tenancy

OpenAPI

Argon2id

Drizzle ORM

TypeBox

JSON Schema validation

Redis

BullMQ

Vitest

Testcontainers

GitHub Actions

CI/CD

Structured logging

Observability
OpenTelemetry
REST APIs
SQL
Transactions
Security
Performance
Load testing (k6)

PROJECT MISSION

Build a production-grade backend for TiendaControl with offline-first event synchronization and immutable ledger architecture.

MILESTONE PLAN

Defined milestones from environment setup and PostgreSQL foundation through authentication, inventory, sales, immutable ledger, cash management, credit ledger, sync protocol, background jobs, DIAN integration architecture, production security, testing maturity, CI/CD, observability, performance/load testing, and production readiness review.

PROJECT OUTCOME

Portfolio-grade backend demonstrating TypeScript, Fastify, PostgreSQL, SQL, transactions, event sourcing, offline synchronization, concurrency control, authentication/authorization, multi-tenancy, REST APIs, testing, Docker, CI/CD, background jobs, observability, security, performance, and production deployment.